

# 16주차 예습과제

📖 딥러닝 파이토치 교과서 8장

<https://colab.research.google.com/drive/1aWaT8LFyxMFvCVoW0DfZUEE4JtBCNIFN#scrollTo=6Him6xRPKdRS>

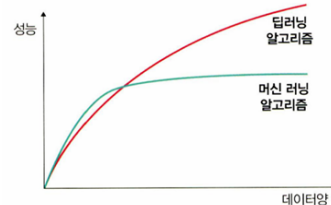
## 8장. 성능 최적화

### 8.1 성능 최적화

#### 8.1.1 데이터를 사용한 성능 최적화

- 최대한 많은 데이터 수집하기: 일반적으로 데이터양이 많을수록 성능이 좋음

▼ 그림 8-1 데이터와 딥러닝, 머신 러닝 알고리즘의 성능 비교



- 데이터 생성하기
- 데이터 범위 조정하기: 활성화 함수(시그모이드, 하이퍼볼릭 탄젠트)로 데이터셋 범위 조정
- + 정규화, 규제화, 표준화

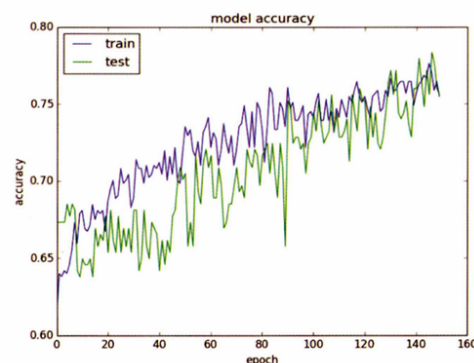
#### 8.1.2 알고리즘을 이용한 성능 최적화

- 유사한 용도의 알고리즘들을 선택하여 모델 훈련, 최적 성능의 알고리즘 선택

#### 8.1.3 알고리즘 튜닝을 위한 성능 최적화

- 진단: 평가 결과를 바탕으로 모델이 과적합인지 혹은 다른 원인으로 성능 향상에 문제가 있는지에 대한 인사이트 얻을 수 있음

▼ 그림 8-2 알고리즘 성능 진단



ex) 훈련 성능이 검증보다 눈에 띄게 좋다? 과적합 → 규제화 진행해 성능 향상

훈련과 검증 결과 모두 성능이 좋지 않다? 과소적합 → 네트워크 구조 변경, 에포크 수 조정

훈련 성능이 검증 넘어서는 변곡점 존재 → 조기 종료



- CUDA(Computed Unified Device Architecture)
  - NVIDIA에서 개발한 GPU 개발 툴
  - 많은 양의 연산을 동시에 처리 가능 → 딥러닝, 채굴 같은 수학적 계산에 사용
- GPU에서 제공하는 코어 정보

▼ 표 8-1 GPU 모델 정보

| 구분           | GEFORCE GTX 1080 | GEFORCE GTX 1070 | GEFORCE GTX 1060(3GB) | GEFORCE GTX 1050 |
|--------------|------------------|------------------|-----------------------|------------------|
| Core         | 2560             | 1920             | 11512                 | 640              |
| Clock(MHz)   | 1607             | 1506             | 1506                  | 1354             |
| Memory       | 11GB             | 8GB              | 6GB                   | 4GB              |
| Memory Speed | 10Gbps           | 8Gbps            | 8Gbps                 | 7Gbps            |

## 8.3 하이퍼파라미터를 이용한 성능 최적화

### 8.3.1 배치 정규화를 이용한 성능 최적화

- 정규화(normalization)
  - 데이터 범위를 사용자가 원하는 범위로 제한하는 것

▼ 그림 8-31 정규화



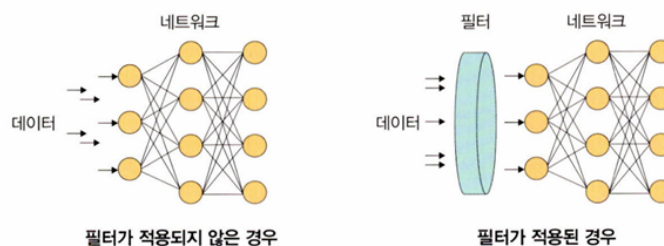
- 각 특성 범위(스케일)를 조정한다는 의미로 특성 스케일링이라고도 함
- 스케일 조정을 위해 `MinMaxScaler()` 기법 사용

$$\frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

(x: 입력 데이터)

- 규제화(regularization)
  - 모델 복잡도를 줄이기 위해 제약을 두는 방법
    - 제약 - 데이터가 네트워크에 들어가기 전에 필터를 적용한 것

▼ 그림 8-32 규제화

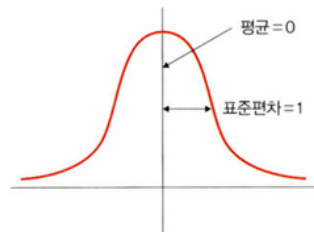


- 드롭아웃, 조기 종료 등이 있음

- 표준화(standardization)

- 기존 데이터를 평균은 0, 표준편차는 1인 형태의 데이터로 만드는 방법
- 표준화 스칼라(standard scaler), z-스코어 정규화(z-score normalization)라고도 함

▼ 그림 8-33 표준화



- 평균을 기준으로 얼마나 떨어져 있는지 살펴볼 때 사용(데이터 분포가 가우시안 분포 따를 때 유용)

$$\frac{x - m}{\sigma}$$

( $\sigma$ : 표준편차,  $x$ : 관측 값(입력 값),  $m$ : 평균)

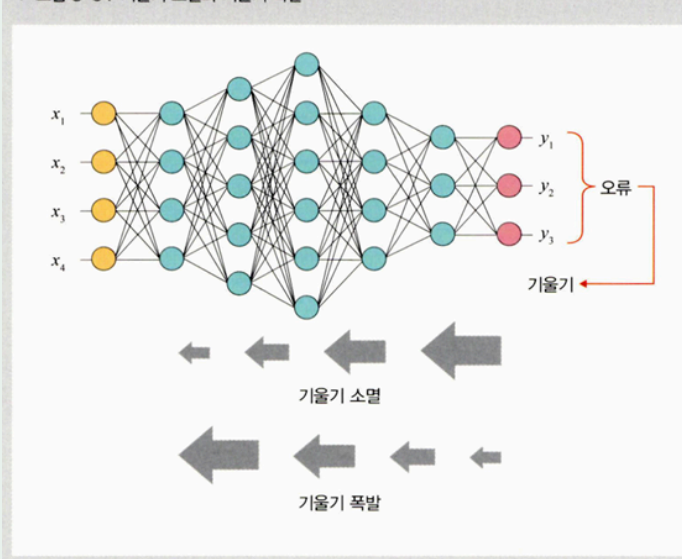
- 배치 정규화(batch normalization)

- 2015년 "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift" 논문에 설명됨
- 데이터 분포가 안정되어 학습 속도 높일 수 있음
- 기울기 소멸이나 기울기 폭발 같은 문제 해결하기 위한 방법
  - 일반적으로 손실 함수로 렐루를 사용하거나 초깃값 튜닝, 학습률 등을 조정

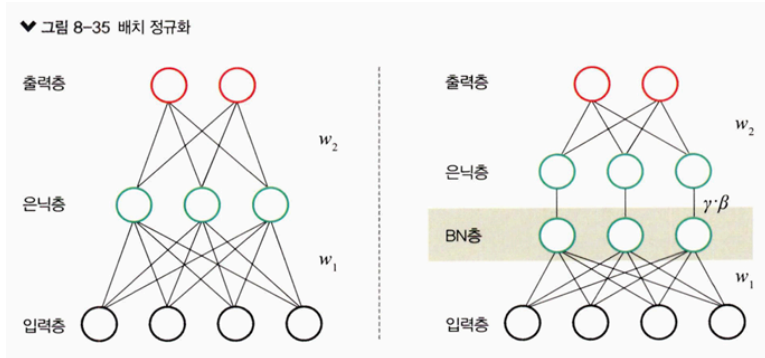
▼ 기울기 소멸과 기울기 폭발

- 기울기 소멸: 오차 정보를 역전파시키는 과정에서 기울기가 급격히 0에 가까워져 학습이 되지 않는 현상
- 기울기 폭발: 학습 과정에서 기울기가 급격히 커지는 현상

▼ 그림 8-34 기울기 소멸과 기울기 폭발







- 기울기 소멸과 폭발 원인은 내부 공변량 변화(internal covariance shift) 때문

- 네트워크의 각 층마다 활성화 함수가 적용되면서 입력 값들의 분포가 계속 바뀌는 현상

→ 분산된 분포를 정규분포로 만들기 위해 표준화와 유사한 방식을 미니 배치에 적용해 평균은 0, 표준편차는 1로 유지하도록 함

$$\mu\beta \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad \cdots \text{①}$$

$$\sigma^2\beta \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu\beta)^2 \quad \cdots \text{②}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu\beta}{\sqrt{\sigma^2\beta + \epsilon}} \quad \cdots \text{③}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \Leftrightarrow BN_{\gamma, \beta}(x_i) \quad \cdots \text{④}$$

$$\left( \begin{array}{l} \text{입력: } \beta = \{x_1, x_2, \dots, x_n\} \\ \text{학습해야 할 하이퍼파라미터: } \gamma, \beta \\ \text{출력: } y_i = BN_{\gamma, \beta}(x_i) \end{array} \right)$$

1. 미니 배치의 분산과 표준편차 구함

2. 미니 배치 평균 구함

3. 정규화 수행

4. 스케일 조정(데이터 분포 조정)

- 매 단계마다 활성화 함수를 거치면서 데이터셋 분포가 일정해지기 때문에 속도를 향상시킬 수 있다

- 단점

1. 배치 크기가 작을 때는 정규화 값이 기존 값과 다른 방향으로 훈련될 수 있다

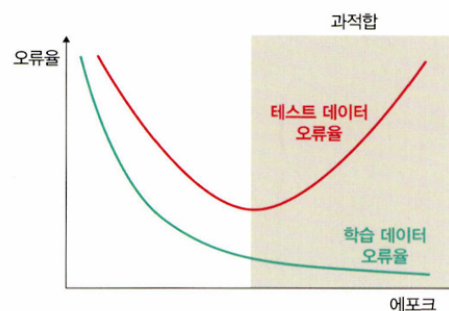
2. RNN은 네트워크 계층별로 미니 정규화를 적용해야 하기 때문에 모델이 더 복잡해지면서 비효율적일 수 있다

→ 가중치 수정, 네트워크 구성 변경 등으로 이러한 문제 해결

### 8.3.2 드롭아웃을 이용한 성능 최적화

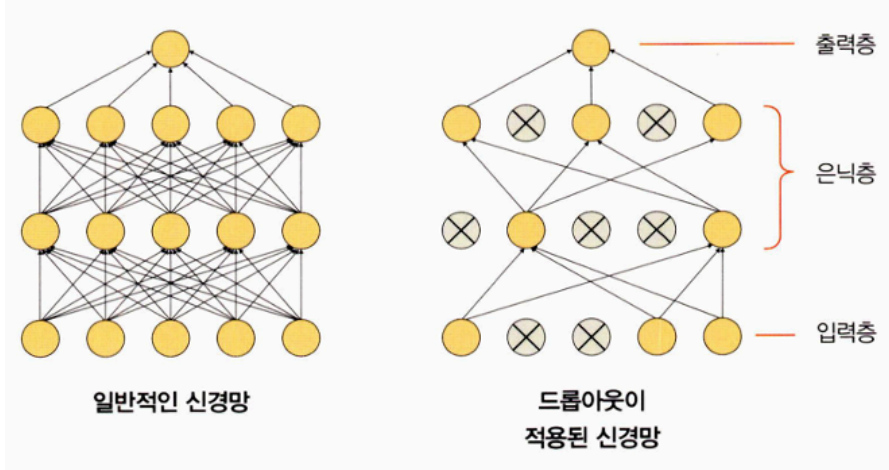
- 과적합: 훈련 데이터셋을 과하게 학습하는 것
- 훈련 데이터셋에 대해 훈련을 계속하여 오류는 줄어들지만 테스트 데이터셋에 대한 오류는 증가하는 모델을 과적합되어 있다고 함

♥ 그림 8-36 훈련과 오류율



- 드롭아웃: 훈련할 때 일정 비율의 뉴런만 사용하고 나머지 뉴런에 해당하는 가중치는 업데이트하지 않는 방법
  - 은닉층에 배치된 노드중 일부를 임의로 끄면서 학습
  - 꺼진 노드는 신호 전달  $X \rightarrow$  지나친 학습을 방지하는 효과

▼ 그림 8-37 드롭아웃



- 어떤 노드를 비활성화할지는 학습할 때마다 무작위로 선정
- 테스트 데이터로 평가할 때는 노드를 모두 사용하여 출력, 노드 삭제 비율(드롭아웃 비율)을 곱해서 성능을 평가
- 훈련 시간이 길어지는 단점이 있음

#### 배치 정규화와 드롭아웃에 대한 파이토치 예제



코드 8-1 라이브러리 호출

```
import torch
import matplotlib.pyplot as plt
import numpy as np

import torchvision
import torchvision.transforms as transforms

import torch.nn as nn
import torch.optim as optim
```



코드 8-2 데이터셋 내려받기

```
trainset = torchvision.datasets.FashionMNIST(
    root='../chap08/data/', train=True,
    download=True,
    transform=transforms.ToTensor())
```



- 내려받은 데이터셋 메모리로 가져오기

#### 코드 8-3 데이터셋을 메모리로 가져오기

```
batch_size = 4
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, shuffle=True)
```

- `trainloader` 변수가 호출될 때 메모리로 가져오게 됨



- 데이터셋을 이미지와 레이블로 분리

#### 코드 8-4 데이터셋 분리

```
dataiter = iter(trainloader)
images, labels = dataiter.next()

print(images.shape)
print(images[0].shape)
print(labels[0].item())
```

```
torch.Size([4, 1, 28, 28])
torch.Size([1, 28, 28])
9
```

```
torch.Size([4, 1, 28, 28])
a b c
```

- 한 번의 배치 크기로 몇 개의 데이터 가져오는지 의미(= `batch_size`)
- 채널 의미 (흑백은 1, 컬러는 3)
- (너비 x 높이) 픽셀 크기 이미지라는 의미



#### 코드 8-5 이미지 데이터를 출력하기 위한 전처리

```
def imshow(img, title):
    plt.figure(figsize=(batch_size * 4, 4)) ----- 출력할 개별 이미지의 크기 지정
    plt.axis('off')
    plt.imshow(np.transpose(img, (1, 2, 0))) ----- ①
```

```
plt.title(title)
plt.show()
```

- 파이토치는 이미지 데이터셋을 [배치 크기, 채널, 너비, 높이] 순서대로 저장  
→ 맷플롯립(matplotlib)으로 출력하기 위해서는 [너비, 높이, 채널] 형태여야 함  
⇒ 데이터의 형태 변경 필요, 넘파이 라이브러리의 `transpose()` 사용



- 이미지 출력을 위한 그래프 방식 정의

#### 코드 8-6 이미지 데이터 출력 함수

```
def show_batch_images(dataloader):
    images, labels = next(iter(dataloader)) ----- 이미지의 크기는 (4, 28, 28, 1(배치 크기, 높이, 너비, 채널))이
                                                    됩니다.

    img = torchvision.utils.make_grid(images) ----- 좌표에 이미지 픽셀을 대응시켜 그리드 형태로 출력합니다.
    imshow(img, title=[str(x.item()) for x in labels]) ----- imshow 함수를 사용함으로써 데이터의
                                                    형태는 (채널, 높이, 너비)에서 (높이, 너비,
                                                    채널)로 변경됩니다.

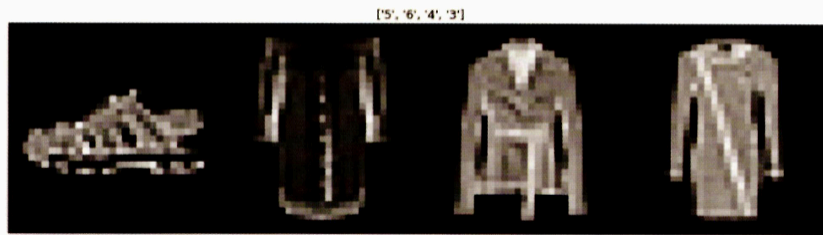
    return images, labels
```



#### 코드 8-7 이미지 출력

```
images, labels = show_batch_images(trainloader)
```

#### ▼ 그림 8-38 FashionMNIST 이미지 샘플



- 표시된 숫자는 클래스(레이블)을 의미

```
classes = {
    0: "T-Shirt/Top",
    1: "Trouser",
    2: "Pullover",
    3: "Dress",
    4: "Coat",
    5: "Sandal",
    6: "Shirt",
    7: "Sneaker",
    8: "Bag",
    9: "Ankle Boot"
}
```



- 모델의 네트워크 구축

코드 8-8 배치 정규화가 적용되지 않은 네트워크

```
class NormalNet(nn.Module):
    def __init__(self):
        super(NormalNet, self).__init__()
        self.classifier = nn.Sequential(
            nn.Linear(784, 48), ----- (28, 28) 크기의 이미지로 입력은 784(28×28) 크기가 됩니다.
            nn.ReLU(),
            nn.Linear(48, 24),
            nn.ReLU(),
            nn.Linear(24, 10) ----- FashionMNIST의 클래스는 총 열 개
        ) ----- nn.Sequential을 사용하면 forward() 함수에서 계층(layer)별로 가독성 있게 코드 구현이 가능

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = self.classifier(x) ----- nn.Sequential에서 정의한 계층 호출
        return x
```



코드 8-9 배치 정규화가 포함된 네트워크

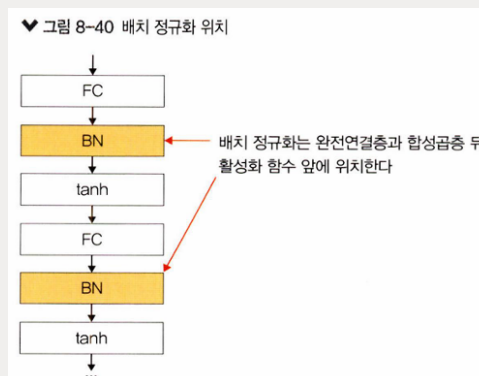
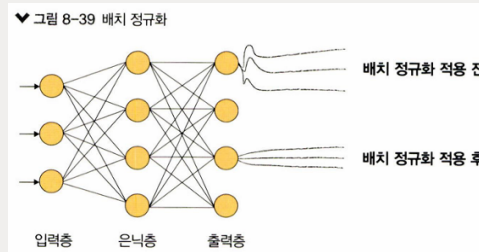
```
class BNNet(nn.Module):
    def __init__(self):
        super(BNNet, self).__init__()
        self.classifier = nn.Sequential(
            nn.Linear(784, 48),
            nn.BatchNorm1d(48), ..... ①
            nn.ReLU(),
            nn.Linear(48, 24),
            nn.BatchNorm1d(24),
            nn.ReLU(),
            nn.Linear(24, 10)
        )

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x
```

## 1. 배치 정규화가 적용되는 부분

`BatchNorm1d` 에서 사용되는 파라미터는 특성 개수로 이전 계층의 출력 채널이 됨

- 배치 정규화를 사용하는 이유
  - 은닉층에서 학습이 진행될 때마다 입력 분포가 변하면서 가중치가 엉뚱한 방향으로 갱신되는 문제 발생하기 때문  
⇒ 신경망의 층이 깊어질수록 학습할 때 가정했던 입력 분포가 변화해 엉뚱한 학습이 진행될 수 있는데, 이를 배치 정규화를 적용해 고르게 맞추어 줌



**코드 8-10** 배치 정규화가 적용되지 않은 모델 선언

```
model = NormalNet()
print(model)
```

```
NormalNet(
  (classifier): Sequential(
    (0): Linear(in_features=784, out_features=48, bias=True)
    (1): ReLU()
    (2): Linear(in_features=48, out_features=24, bias=True)
    (3): ReLU()
    (4): Linear(in_features=24, out_features=10, bias=True)
  )
)
```

**코드 8-11** 배치 정규화가 적용된 모델 선언

```
model_bn = BNNet()
print(model_bn)
```

```
BNNet(
  (classifier): Sequential(
    (0): Linear(in_features=784, out_features=48, bias=True)
    (1): BatchNorm1d(48, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): Linear(in_features=48, out_features=24, bias=True)
    (4): BatchNorm1d(24, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (5): ReLU()
    (6): Linear(in_features=24, out_features=10, bias=True)
  )
)
```



- 데이터로더를 이용하여 앞에서 내려받았던 데이터셋을 메모리로 불러오기
  - 앞에서는 이미지 출력을 위한 용도로 배치 크기 4로 설정
  - 이번에는 학습을 위한 용도로 배치 크기를 512로 지정

**코드 8-12** 데이터셋 메모리로 불러오기

```
batch_size = 512
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, shuffle=True)
```

**코드 8-13 옵티마이저, 손실 함수 지정**

```
loss_fn = nn.CrossEntropyLoss()  
opt = optim.SGD(model.parameters(), lr=0.01)  
opt_bn = optim.SGD(model_bn.parameters(), lr=0.01)
```





#### 코드 8-14 모델 학습

```
loss_arr = []
loss_bn_arr = []
max_epochs = 2

for epoch in range(max_epochs):
    for i, data in enumerate(trainloader, 0):

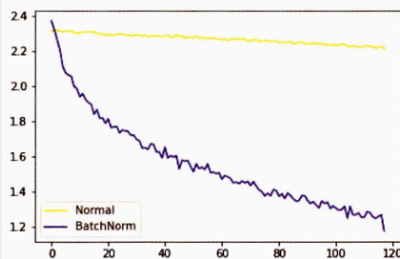
        inputs, labels = data
        opt.zero_grad() ----- 배치 정규화가 적용되지 않은 모델의 학습
        outputs = model(inputs)
        loss = loss_fn(outputs, labels)
        loss.backward()
        opt.step()

        opt_bn.zero_grad() ----- 배치 정규화가 적용된 모델의 학습
        outputs_bn = model_bn(inputs)
        loss_bn = loss_fn(outputs_bn, labels)
        loss_bn.backward()
        opt_bn.step()

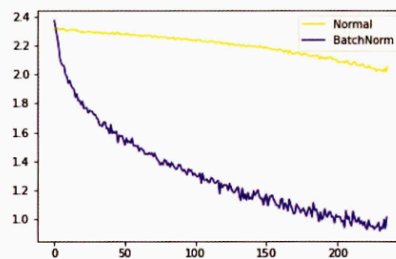
        loss_arr.append(loss.item())
        loss_bn_arr.append(loss_bn.item())

plt.plot(loss_arr, 'yellow', label='Normal')
plt.plot(loss_bn_arr, 'blue', label='BatchNorm')
plt.legend()
plt.show()
```

♥ 그림 8-41 첫 번째 에포크에 대한 학습 결과

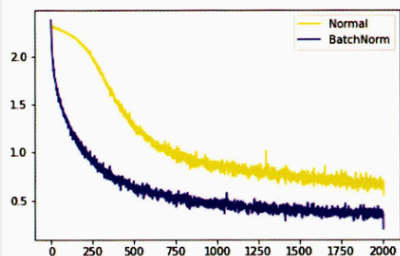


♥ 그림 8-42 두 번째 에포크에 대한 학습 결과

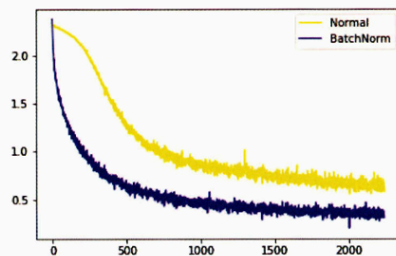


... 중간 생략 ...

♥ 그림 8-43 18번째 에포크에 대한 학습 결과



♥ 그림 8-44 19번째 에포크에 대한 학습 결과



- 두 모델 모두 시간이 흐를수록 오차가 줄어드는 것 확인
- 배치 정규화가 적용된 모델의 경우 더 낮은 값으로 안정적인 범위 내에서 줄어들고 있음  
⇒ 배치 정규화가 적용된 모델은 에포크가 진행될수록 오차도 줄어들면서 안정적인 학습을 하고 있음

---



#### 코드 8-15 데이터셋의 분포를 출력하기 위한 전처리

```

N = 50
noise = 0.3

x_train = torch.unsqueeze(torch.linspace(-1, 1, N), 1) -----①
y_train = x_train + noise * torch.normal(torch.zeros(N, 1), torch.ones(N, 1)) -----②

x_test = torch.unsqueeze(torch.linspace(-1, 1, N), 1)
y_test = x_test + noise * torch.normal(torch.zeros(N, 1), torch.ones(N, 1))

```

### 1. 훈련 데이터셋이 -1~1의 값을 갖도록 조정

```

x_train = torch.unsqueeze(torch.linspace(-1, 1, N), 1)

```

①

---

②

- a. `torch.linspace` : 주어진 범위에서 균등한 값을 갖는 텐서를 만들기 위해 사용  
`torch.linspace(-1, 1, N)` 은 -1~1의 범위에서 N개의 균등한 값을 갖는 텐서를 생성

#### ▼ `torch.linspace` 사용 예시

```

import torch
print(torch.linspace(0, 10)) ----- 0~10을 100으로 분할
print('-----')
print(torch.linspace(0, 10, 5)) ----- 0~10을 5로 분할

```

```

tensor([ 0.0000,  0.1010,  0.2020,  0.3030,  0.4040,  0.5051,  0.6061,  0.7071,  0.8081,
  0.9091,  1.0101,  1.1111,  1.2121,  1.3131,  1.4141,  1.5152,  1.6162,  1.7172,  1.8182,
  1.9192,  2.0202,  2.1212,  2.2222,  2.3232,  2.4242,  2.5253,  2.6263,  2.7273,  2.8283,
  2.9293,  3.0303,  3.1313,  3.2323,  3.3333,  3.4343,  3.5354,  3.6364,  3.7374,  3.8384,
  3.9394,  4.0404,  4.1414,  4.2424,  4.3434,  4.4444,  4.5455,  4.6465,  4.7475,  4.8485,
  4.9495,  5.0505,  5.1515,  5.2525,  5.3535,  5.4545,  5.5556,  5.6566,  5.7576,  5.8586,
  5.9596,  6.0606,  6.1616,  6.2626,  6.3636,  6.4646,  6.5657,  6.6667,  6.7677,  6.8687,
  6.9697,  7.0707,  7.1717,  7.2727,  7.3737,  7.4747,  7.5758,  7.6768,  7.7778,  7.8788,
  7.9798,  8.0808,  8.1818,  8.2828,  8.3838,  8.4848,  8.5859,  8.6869,  8.7879,  8.8889,
  8.9899,  9.0909,  9.1919,  9.2929,  9.3939,  9.4950,  9.5960,  9.6970,  9.7980,  9.8990,
  10.0000])
-----
tensor([ 0.0000,  2.5000,  5.0000,  7.5000, 10.0000])

```

- b. `torch.unsqueeze` : 차원을 늘리기 위해 사용  
`torch.unsqueeze(torch.linspace(-1, 1, N), 1)` 은 `torch.linspace(-1, 1, N)` 텐서의 첫 번째 자리에 차원을 증가시키겠다는 의미

### 2. 훈련 데이터셋 값의 범위가 정규분포를 갖도록 조정

```

y_train = x_train + noise * torch.normal(torch.zeros(N, 1), torch.ones(N, 1))

```

①                      ②                      ③

- a. `torch.normal` : 정규분포로부터 무작위 표본 추출을 위해 사용  
`torch.normal` (평균, 표준편차)를 의미(평균은 0, 표준편차는 1이 기본값)
- b. `torch.zeros` : 0 값을 갖는 Nx1 텐서를 생성
- c. `torch.ones` : 1 값을 갖는 Nx1 텐서를 생성



#### 코드 8-16 데이터 분포를 그래프로 출력

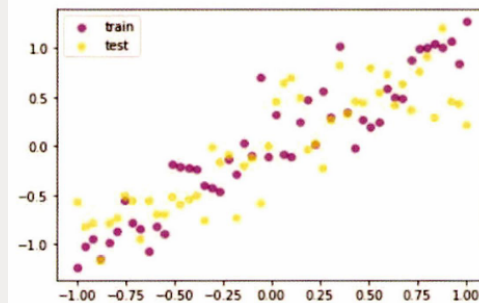
```
plt.scatter(x_train.data.numpy(), y_train.data.numpy(), c='purple',  
            alpha=0.5, label='train') ..... ①  
plt.scatter(x_test.data.numpy(), y_test.data.numpy(), c='yellow',  
            alpha=0.5, label='test')  
plt.legend()  
plt.show()
```

1. `plt.scatter()` 는 데이터르 그래프상에 점으로 출력해서 데이터 분포를 확인하고자 할 때 사용

```
plt.scatter(x_train.data.numpy(), y_train.data.numpy(), c='purple',  
            ④          ⑤          ③  
            alpha=0.5, label='train')  
            ④          ⑤
```

- a. 첫 번째 파라미터: x축에 위치할 데이터
- b. 두 번째 파라미터: y축에 위치할 데이터
- c. c: 그래프로 출력되는 마커의 색상
- d. alpha: 마커에 대한 투명도, 1이면 완전 불투명 상태
- e. label: 맷플롯립의 레전드와 같은 역할, `plt.legend()` 와 함께 사용되어야 함

▼ 그림 8-45 FashionMNIST 데이터셋의 분포



- 훈련과 테스트 데이터가 고르게 분포되어 있는 것을 확인할 수 있음

**코드 8-17 드롭아웃을 위한 모델 생성**

```
N_h = 100
model = torch.nn.Sequential(
    torch.nn.Linear(1, N_h),
    torch.nn.ReLU(),
    torch.nn.Linear(N_h, N_h),
    torch.nn.ReLU(),
    torch.nn.Linear(N_h, 1),
) ----- 드롭아웃이 적용되지 않은 모델

model_dropout = torch.nn.Sequential(
    torch.nn.Linear(1, N_h),
    torch.nn.Dropout(0.2), ----- 드롭아웃 적용
    torch.nn.ReLU(),
    torch.nn.Linear(N_h, N_h),
    torch.nn.Dropout(0.2),
    torch.nn.ReLU(),
    torch.nn.Linear(N_h, 1),
) ----- 드롭아웃이 적용된 모델
```

**코드 8-18 옵티마이저와 손실 함수 지정**

```
opt = torch.optim.Adam(model.parameters(), lr=0.01)
opt_dropout = torch.optim.Adam(model_dropout.parameters(), lr=0.01)
loss_fn = torch.nn.MSELoss()
```



#### 코드 8-19 모델 학습

```
max_epochs = 1000
for epoch in range(max_epochs):
    pred = model(x_train) ----- 드롭아웃이 적용되지 않은 모델 학습
    loss = loss_fn(pred, y_train)
    opt.zero_grad()
    loss.backward()

    opt.step()

    pred_dropout = model_dropout(x_train) ----- 드롭아웃이 적용된 모델 학습
    loss_dropout = loss_fn(pred_dropout, y_train)
    opt_dropout.zero_grad()
    loss_dropout.backward()
    opt_dropout.step()

    if epoch % 50 == 0: ----- epoch를 50으로 나눈 나머지가 0이면 다음 진행
        model.eval()
        model_dropout.eval()

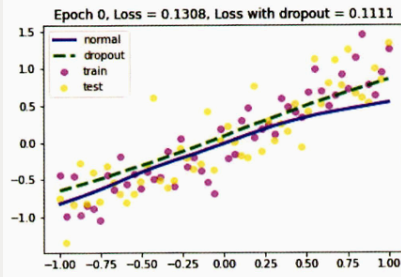
        test_pred = model(x_test)
        test_loss = loss_fn(test_pred, y_test)

        test_pred_dropout = model_dropout(x_test)
        test_loss_dropout = loss_fn(test_pred_dropout, y_test)

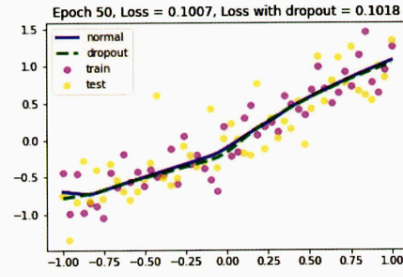
        plt.scatter(x_train.data.numpy(), y_train.data.numpy(), c='purple',
                    alpha=0.5, label='train')
        plt.scatter(x_test.data.numpy(), y_test.data.numpy(), c='yellow',
                    alpha=0.5, label='test')
        plt.plot(x_test.data.numpy(), test_pred.data.numpy(), 'b-', lw=3,
                 label='normal') ----- 파란색 실선으로 x축은 테스트 데이터셋, y축은 드롭아웃이
                                     적용되지 않은 모델의 결과를 그래프로 출력
        plt.plot(x_test.data.numpy(), test_pred_dropout.data.numpy(), 'g--', lw=3,
                 label='dropout') ----- 초록색 점선으로 x축은 테스트 데이터셋, y축은 드롭아웃이
                                     적용된 모델의 결과를 그래프로 출력

        plt.title('Epoch %d, Loss=%0.4f, Loss with dropout=%0.4f' %
                  (epoch, test_loss, test_loss_dropout)) ----- 에포크, 드롭아웃이 적용되지 않은
                                                                모델의 오차, 드롭아웃이 적용된
                                                                모델의 오차를 타이틀로 출력
        plt.legend()
        model.train()
        model_dropout.train()
        plt.pause(0.05)
```

♥ 그림 8-46 첫 번째 드롭아웃에 대한 학습 결과

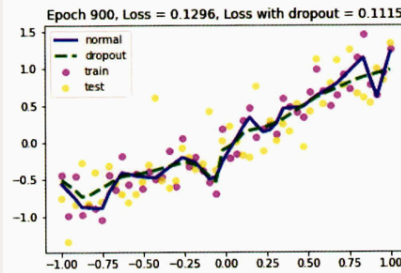


♥ 그림 8-47 두 번째 드롭아웃에 대한 학습 결과

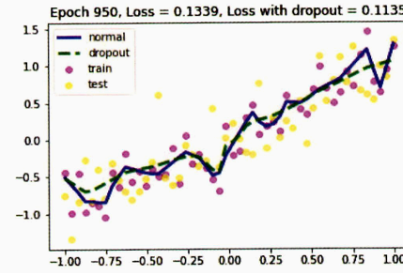


... 중간 생략 ...

♥ 그림 8-48 19번째 드롭아웃에 대한 학습 결과



♥ 그림 8-49 20번째 드롭아웃에 대한 학습 결과

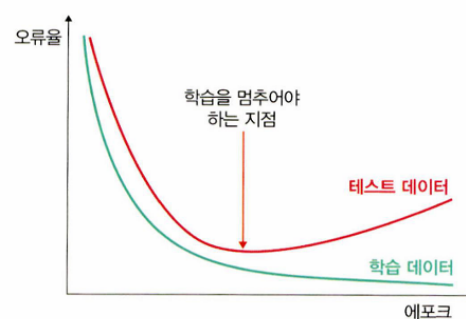


- 드롭아웃을 적용했을 때의 오차가 더 낮은 것을 확인
- 훈련 횟수가 늘어날수록 드롭아웃이 적용되지 않은 모델은 과적합 현상을 보임
- 드롭아웃은 과적합 현상을 방지하기 위해 사용, 드롭아웃이 적용된 모델은 과적합 현상이 발생하지 않는 것을 확인할 수 있음

### 8.3.3 조기 종료를 이용한 성능 최적화

- 조기 종료(early stopping)
    - 뉴럴 네트워크가 과적합을 회피하는 규제 기법
    - 훈련 데이터와 별도로 검증 데이터를 준비, 매 에포크마다 검증 데이터에 대한 오차를 측정해 모델의 종료 시점을 제어
- 과적합이 발생하기 전까지 학습에 대한 오차와 검증에 대한 오차 모두 감소하지만, 과적합이 발생하면 훈련 데이터셋에 대한 오차는 감소하는 반면 검증 데이터셋에 대한 오차는 증가
- ⇒ 조기 종료는 검증 데이터셋에 대한 오차가 증가하는 시점에서 학습을 멈추도록 조정

♥ 그림 8-50 조기 종료



- 조기 종료는 학습 종료 시점을 결정할 뿐이지 최고의 성능을 갖는 모델을 보장하진 않는다는 점에 주의

#### 조기 종료와 학습률을 조정해 성능 향상시키는 예제

**코드 8-20 라이브러리 호출**

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.models as models ----- 사전 학습된 모델을 이용하고자 할 때 사용하는 라이브러리
from torchvision import transforms, datasets

import matplotlib
import matplotlib.pyplot as plt
import time

import argparse
from tqdm import tqdm
matplotlib.style.use('ggplot') ----- 출력 그래프에서 격자로 숫자 범위가 눈에 잘 띄도록 하는 스타일
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```



- 데이터 크기 조정 및 데이터 정규화 등의 전처리

**코드 8-21 데이터셋 전처리**

```
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])
val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])
```

**코드 8-22 데이터셋 가져오기**

```
train_dataset = datasets.ImageFolder(
    root=r'../chap08/data/archive/train',
    transform=train_transform
)
train_dataloader = torch.utils.data.DataLoader(
    train_dataset, batch_size=32, shuffle=True,
)
val_dataset = datasets.ImageFolder(
    root=r'../chap08/data/archive/test',
    transform=val_transform
)
val_dataloader = torch.utils.data.DataLoader(
    val_dataset, batch_size=32, shuffle=False,
)
```





- 사전 학습된 ResNet50을 사용해 모델 생성

#### 코드 8-23 모델 생성

```
def resnet50(pretrained=True, requires_grad=False):  
    model = models.resnet50(progress=True, pretrained=pretrained)  
    if requires_grad == False: ..... 파라미터를 고정하여 backward() 중에 기울기가 계산되지 않도록 합니다.  
        for param in model.parameters(): requires_grad=False를 파라미터로 받았기 때문에 해당 구문이 실행됩니다.  
            param.requires_grad = False  
    elif requires_grad == True: ..... 파라미터 값이 backward() 중에 기울기 계산에 반영됩니다.  
        for param in model.parameters():  
            param.requires_grad = True  
    model.fc = nn.Linear(2048, 2) ..... 마지막 분류를 위한 계층은 학습을 진행합니다.  
    return model
```



## • 학습률 조정

- 학습이 진행되는 과정에서 학습률을 조금씩 낮추어 주는 성능 튜닝 기법
- 학습률 감소는 학습률 스케줄러를 이용
  - 주어진 'patience' 횟수만큼 검증 데이터셋에 대한 오차 감소가 없으면 주어진 'factor'만큼 학습률을 감소시켜 모델 학습의 최적화가 가능하도록 도와줌

코드 8-24 학습률 감소

```
class LRScheduler():
    def __init__(
        self, optimizer, patience=5, min_lr=1e-6, factor=0.5
    ):
        self.optimizer = optimizer
        self.patience = patience
        self.min_lr = min_lr
        self.factor = factor
        self.lr_scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
            self.optimizer,
            mode='min',
            patience=self.patience,
            factor=self.factor,
            min_lr=self.min_lr,
            verbose=True
        ) ----- ①
```

```
def __call__(self, val_loss):
    self.lr_scheduler.step(val_loss) ----- ②
```

### 1. 학습 과정에서 모델 성능에 대한 개선이 없을 경우 학습률 값을 조절하여 모델의 개선을 유도하는 콜백 함수

```
torch.optim.lr_scheduler.ReduceLROnPlateau(self.optimizer, mode='min',
                                           patience=self.patience, factor=self.factor, min_lr=self.min_lr, verbose=True)
                                           ④          ⑤          ⑥          ⑦          ⑧
```

- `lr_scheduler.ReduceLROnPlateau` : 검증 데이터셋에 대한 오차의 변동이 없으면 학습률을 factor배로 감소시킴
- `optimizer` : 파라미터(가중치)를 갱신시키는 부분
- `mode` : 언제 학습률을 조정할지에 대한 기준이 되는 값
- `patience` : 학습률을 업데이트하기 전에 몇 번의 에포크를 기다려야 하는지 결정
- `factor` : 학습률을 얼마나 감소시킬지 지정하는 파라미터. 새로운 학습률은 기존 학습률 \* factor가 됨
- `min_lr` : 학습률의 하한선을 지정
- `verbose` : 초기 종료의 시작과 끝을 출력. 1로 설정할 경우 초기 종료가 적용되면 적용되었다고 출력됨.

#### ▼ 콜백 함수

- 개발자가 명시적으로 함수를 호출하는 것이 아닌, 함수 등록만 하고 특정 이벤트 발생에 의해 함수를 호출하고 처리하도록 하는 것
  - 동기적 함수: 코드가 위에서 아래로, 왼쪽에서 오른쪽으로 순차적으로 실행되는 함수
  - 비동기적 함수: 병렬 처리적 함수. 어떤 코드 실행 시 시간이 걸리면 해당 코드가 완료되는 걸 기다리지 않고 다른 코드가 먼저 처리되도록 함
- 2. 실제로 학습률을 업데이트. 에포크 단위로 검증 데이터셋에 대한 오차를 받아 이전 오차와 비교했을 때 차이가 없다면 학습률을 업데이트 함



- 특정 에포크 후에도 오차가 개선되지 않을 때 훈련을 조기 종료

코드 8-25 조기 종료

```
class EarlyStopping():
    def __init__(self, patience=5, verbose=False, delta=0,
                 path='../chap08/data/checkpoint.pt'):
        self.patience = patience ①
        self.verbose = verbose
        self.counter = 0
        self.best_score = None ② ----- 검증 데이터셋에 대한 오차 최소화 값(오차가 가장 낮은 값)
        self.early_stop = False ----- 조기 종료를 의미하며 초깃값은 False로 설정
        self.val_loss_min = np.Inf ----- np.Inf(infinity)는 넘파이에서 무한대를 표현
        self.delta = delta ----- ②
        self.path = path ----- 모델이 저장될 경로

    def __call__(self, val_loss, model): ----- 에포크만큼 학습이 반복되면서 best_loss가 갱신되고,
                                                best_loss에 진전이 없으면 조기 종료한 후 모델을 저장
        score = -val_loss
        if self.best_score is None: ----- best_score에 값이 존재하지 않으면 실행
            self.best_score = score
            self.save_checkpoint(val_loss, model)
        elif score < self.best_score + self.delta: ----- best_score + delta가 score보다 크면 실행
            self.counter += 1
            print(f'EarlyStopping counter: {self.counter} out of {self.patience}')
            if self.counter >= self.patience:
                self.early_stop = True
        else: ----- 그 외 모든 경우에 실행
            self.best_score = score
            self.save_checkpoint(val_loss, model)
            self.counter = 0

    def save_checkpoint(self, val_loss, model): ----- 검증 데이터셋에 대한 오차가 감소하면 모델을 저장
        if self.verbose:
            print(f'Validation loss decreased ({self.val_loss_min:.6f} --> {val_loss:.6f}). Saving model ...')
        torch.save(model.state_dict(), self.path) ----- 지정된 경로에 모델 저장
        self.val_loss_min = val_loss
```

1. **patience**: 오차 개선이 없다고 바로 종료하지 않고 개선이 없는 에포크를 얼마나 기다려 줄지 지정
  2. **delta**: 오차가 개선되고 있다고 판단하기 위한 최소 변화량
- 케라스에서 제공하는 콜백(**keras.callbacks**)을 이용하면 쉽게 조기 종료 구현 가능하지만 파이토치에서는 직접 구현해야 함

```
from keras.callbacks import ModelCheckpoint, EarlyStopping
checkpoint = ModelCheckpoint('checkpoint-epoch.h5', format(EPOCH, BATCH_SIZE), ----- 파일명 지정
                            monitor='val_loss', ----- val_loss 값이 개선되었을 때 호출
                            verbose=1, ----- 로그 출력
                            save_best_only=True, ----- 가장 최적의 값만 저장
                            mode='auto' ----- auto 의미는 시스템이 알아서 best 값을 찾으라는 것
                            )

earlystopping = EarlyStopping(monitor='val_loss', ----- 학습률 업데이트 기준 설정(val_loss)
                              patience=10 ----- 에포크가 진행되는 열 번 동안 모델의 오차가
                              ) ----- 개선되지 않으면 종료
```



- 학습률 감소와 조기 종료 두 개에 대한 성능 튜닝 → 인수 값에 따라 다른 동작 하도록 `argparse` 라이브러리 사용
  - `ArgumentParser()` 이용해 변수와 타입 정의
  - `add_argument()` 이용해 변수에 인수 값을 하나씩 추가
  - `parse_args()` 를 통해 사용자로부터 입력받은 값을 `args` 변수에 저장

#### 코드 8-26 인수 값 지정

```
parser = argparse.ArgumentParser() ----- 인수 값을 받을 수 있는 인스턴스 생성
parser.add_argument('--lr-scheduler', dest='lr_scheduler', action='store_true') ----- ①
parser.add_argument('--early-stopping', dest='early_stopping', action='store_true') ----- ②
args = vars(parser.parse_args()) ----- ③
```

1. 원하는 인수 값 추가. 이때 `parser.add_argument()` 는 인수 개수만큼 만들어 줌

```
parser.add_argument('--lr-scheduler', dest='lr_scheduler', action='store_true')
                    (a)                (b)                (c)
```

- a. 첫 번째 파라미터: 옵션 문자열의 이름으로 명령을 실행할 때 사용  
ex) `>python main.py --lr-scheduler`
  - b. `dest`: 입력 값이 저장되는 변수
  - c. `action`: `store_true` 로 지정하면 입력 값을 `dest` 파라미터에 의해 생성된 변수에 저장
2. 입력받은 인수 값이 실제로 `args` 변수에 저장됨



- 사전 훈련된 ResNet50의 파라미터 확인
- ipywidgets 라이브러리 설치
  - `pip install ipywidgets`

#### 코드 8-27 사전 훈련된 모델의 파라미터 확인

```
print(f"Computation device: {device}\n") ----- CPU를 사용하는지 GPU를 사용하는지 검사
model = models.resnet50(pretrained=True).to(device) ----- 사전 훈련된 ResNet50 사용
total_params = sum(p.numel() for p in model.parameters()) ----- 총 파라미터 수
print(f"{total_params:,} total parameters.")
```

```
total_trainable_params = sum(
    p.numel() for p in model.parameters() if p.requires_grad) ----- 학습 가능한 파라미터 수
print(f"{total_trainable_params:,} training parameters.")
```

Computation device: cpu

25,557,032 total parameters.  
25,557,032 training parameters.

**코드 8-28** 옵티마이저와 손실 함수 지정

```
lr = 0.001
epochs = 100
optimizer = optim.Adam(model.parameters(), lr=lr)
criterion = nn.CrossEntropyLoss()
```

**코드 8-29** 오차, 정확도 및 모델의 이름에 대한 문자열

```
loss_plot_name = 'loss' ----- 오차 출력에 대한 문자열
acc_plot_name = 'accuracy' ----- 정확도 출력에 대한 문자열
model_name = 'model' ----- 모델을 저장하기 위한 문자열
```

**코드 8-30** 오차, 정확도 및 모델의 이름에 대한 문자열

```
if args['lr_scheduler']:
    print('INFO: Initializing learning rate scheduler')
    lr_scheduler = LRScheduler(optimizer)

loss_plot_name = 'lrs_loss' ----- 학습률 감소를 적용했을 때의 오차에 대한 문자열
acc_plot_name = 'lrs_accuracy' ----- 학습률 감소를 적용했을 때의 정확도에 대한 문자열
model_name = 'lrs_model' ----- 학습률 감소를 적용했을 때의 모델에 대한 문자열

if args['early_stopping']:
    print('INFO: Initializing early stopping')
    early_stopping = EarlyStopping()
    loss_plot_name = 'es_loss' ----- 조기 종료를 적용했을 때의 오차에 대한 문자열
    acc_plot_name = 'es_accuracy' ----- 조기 종료를 적용했을 때의 정확도에 대한 문자열
    model_name = 'es_model' ----- 조기 종료를 적용했을 때의 모델에 대한 문자열
```



#### 코드 8-31 모델 학습 함수

```
def training(model, train_dataloader, train_dataset, optimizer, criterion):
    print('Training')
    model.train()
    train_running_loss = 0.0
    train_running_correct = 0
    counter = 0
    total = 0
    prog_bar = tqdm(enumerate(train_dataloader), total=int(len(train_dataset)/
        train_dataloader.batch_size)) ----- 훈련 진행 과정을 시각적으로 표현
    for i, data in prog_bar:
        counter += 1
        data, target = data[0].to(device), data[1].to(device)
        total += target.size(0)
        optimizer.zero_grad()
        outputs = model(data)
        loss = criterion(outputs, target)
        train_running_loss += loss.item()
        _, preds = torch.max(outputs.data, 1)
        train_running_correct += (preds == target).sum().item()
        loss.backward()
        optimizer.step()

    train_loss = train_running_loss / counter
    train_accuracy = 100. * train_running_correct / total
    return train_loss, train_accuracy
```



#### 코드 8-32 모델 검증 함수

```
def validate(model, test_dataloader, val_dataset, criterion):
    print('Validating')
    model.eval()
    val_running_loss = 0.0
    val_running_correct = 0
    counter = 0
    total = 0
    prog_bar = tqdm(enumerate(test_dataloader), total=int(len(val_dataset)/
        test_dataloader.batch_size)) ----- 모델 검증 과정을 시각적으로 표현
    with torch.no_grad():
        for i, data in prog_bar:
            counter += 1
            data, target = data[0].to(device), data[1].to(device)
            total += target.size(0)
            outputs = model(data)
            loss = criterion(outputs, target)

            val_running_loss += loss.item()
            _, preds = torch.max(outputs.data, 1)
            val_running_correct += (preds == target).sum().item()

    val_loss = val_running_loss / counter
    val_accuracy = 100. * val_running_correct / total
    return val_loss, val_accuracy
```



### 코드 8-33 모델 학습

train\_loss, train\_accuracy = [], [] ----- 훈련 데이터셋을 이용한 모델 학습 결과(오차, 정확도)를  
저장하기 위한 변수(리스트 형태를 갖습니다)

val\_loss, val\_accuracy = [], [] ----- 검증 데이터셋을 이용한 모델 성능 결과(오차, 정확도)를  
저장하기 위한 변수(리스트 형태를 갖습니다)

```
start = time.time()
for epoch in range(epochs):
    print(f"Epoch {epoch+1} of {epochs}")
    train_epoch_loss, train_epoch_accuracy = training(
        model, train_dataloader, train_dataset, optimizer, criterion
    )
    val_epoch_loss, val_epoch_accuracy = validate(
        model, val_dataloader, val_dataset, criterion
    )

    train_loss.append(train_epoch_loss)
    train_accuracy.append(train_epoch_accuracy)
    val_loss.append(val_epoch_loss)
    val_accuracy.append(val_epoch_accuracy)
    if args['lr_scheduler']: ----- 인수 값이 lr_scheduler이면 다음을 실행
        lr_scheduler(val_epoch_loss)
    if args['early_stopping']: ----- 인수 값이 early_stopping이면 다음을 실행
        early_stopping(val_epoch_loss, model)
        if early_stopping.early_stop:
            break
    print(f"Train Loss: {train_epoch_loss:.4f}, Train Acc: {train_epoch_accuracy:.2f}")
    print(f"Val Loss: {val_epoch_loss:.4f}, Val Acc: {val_epoch_accuracy:.2f}")
end = time.time()
print(f"Training time: {(end-start)/60:.3f} minutes")
```

```
Epoch 1 of 100
Training
Validating
Train Loss: 2.1123, Train Acc: 60.04
Val Loss: 13.3437, Val Acc: 55.20
Epoch 2 of 100
Training
Validating
Train Loss: 0.5598, Train Acc: 73.90
Val Loss: 0.6033, Val Acc: 68.40
... 중간 생략 ...
Epoch 99 of 100
Training
Validating
Train Loss: 0.0340, Train Acc: 99.20
Val Loss: 1.5805, Val Acc: 68.60
Epoch 100 of 100
Training
Validating
Train Loss: 0.0394, Train Acc: 98.80
Val Loss: 1.0129, Val Acc: 76.00
Training time: 654.208 minutes
```



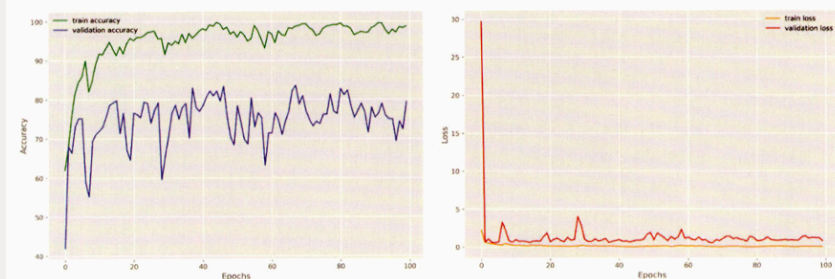


#### 코드 8-34 모델 학습 결과 출력

```
print('Saving loss and accuracy plots...')
plt.figure(figsize=(10, 7))
plt.plot(train_accuracy, color='green', label='train accuracy') ..... 훈련 데이터셋에 대한 정확도를 그래프로 출력
plt.plot(val_accuracy, color='blue', label='validation accuracy') ..... 검증 데이터셋에 대한 정확도를 그래프로 출력
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.savefig(f"../chap08/img/{acc_plot_name}.png")
plt.show()
plt.figure(figsize=(10, 7))
plt.plot(train_loss, color='orange', label='train loss') ..... 훈련 데이터셋에 대한 오차를 그래프로 출력
plt.plot(val_loss, color='red', label='validation loss') ..... 검증 데이터셋에 대한 오차를 그래프로 출력
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.savefig(f"../chap08/img/{loss_plot_name}.png")
plt.show()

print('Saving model...')
torch.save(model.state_dict(), f"../chap08/img/{model_name}.pth") ..... 모델을 저장
print('TRAINING COMPLETE')
```

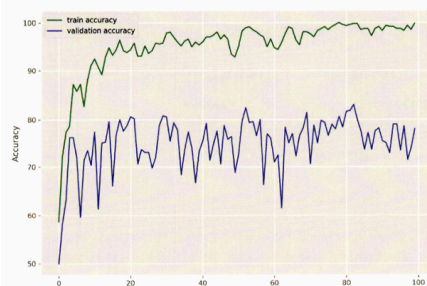
▼ 그림 8-53 출력 결과



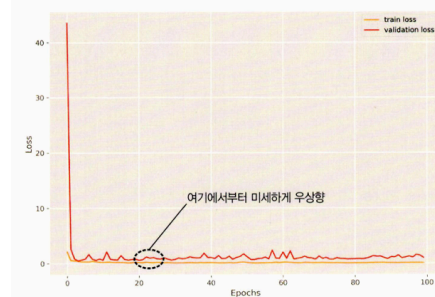
⇒ 코드를 .py 파일로 저장 후 인수 적용 별 실행 결과 확인

- 어떤 인수도 적용하지 않았을 때
  - 정확도와 오차 그래프의 변동 심함
  - 검증 데이터셋에 대한 오차가 미세한 차이로 우상향 → 모델이 과적합되기 시작
  - 적절한 훈련을 계속하려면 학습률 값을 줄여야 함

▼ 그림 8-54 어떤 인수도 적용되지 않았을 때의 정확도

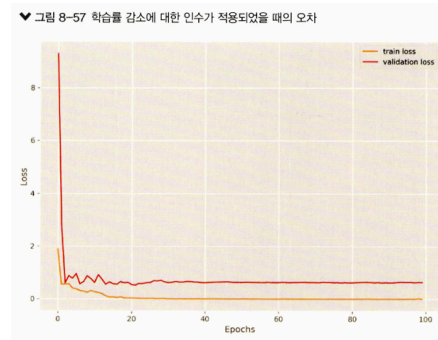
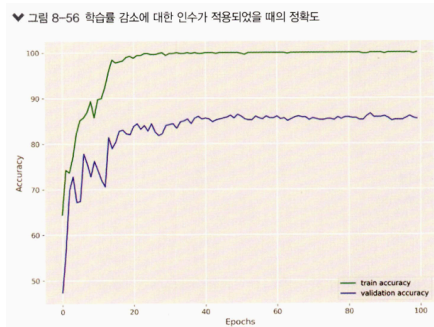


▼ 그림 8-55 어떤 인수도 적용되지 않았을 때의 오차





- 학습률 감소와 관련된 인수 입력했을 때
  - 정확도 그래프가 완만한 곡선 형태 보이고 검증 데이터셋에 대한 정확도도 높게 나타남
  - 오차 그래프의 우상향 경향도 사라짐 → 학습률 스케줄러가 성능 향상에 기여



- 조기 종료 인수 입력했을 때
  - 다섯 번째 에포크부터 조기 종료가 수행됨 (실제 학습은 네 번째 에포크까지만 수행)
  - 조기 종료가 항상 성능에 좋은 영향 미치는 것은 아니기 때문에 적절히 적용해야 함

